
Sprawozdanie z Laboratorium: Bazy Danych

Paweł Łoćwin

Jun 10, 2026

CONTENTS

1	Rozdział 1: Kontrola i konserwacja	2
1.1	Wprowadzenie	2
1.2	Zarządzanie przestrzenią i mechanizm współbieżności	2
1.3	Optymalizacja i statystyki planisty zapytań	2
1.4	Bezpieczeństwo i rygorystyczna kontrola dostępu	3
1.5	Strategie zabezpieczania przed utratą danych	3
1.6	Podsumowanie	3
2	Badania literaturowe	5
2.1	Sprzęt dla bazy danych	5
2.2	Konfiguracja bazy danych PostgreSQL	5
2.2.1	Wstęp	5
2.2.2	Podstawowe parametry	5
2.2.3	Lokalizacja i struktura katalogów PostgreSQL	5
2.2.4	Środowiska i profile	6
2.2.5	Automatyzacja	6
2.2.6	Przykłady	6
2.2.7	Najlepsze praktyki	7
2.2.8	Podsumowanie	7
2.2.9	Szczegółowe omówienie plików konfiguracyjnych	7
2.2.10	postgresql.conf	7
2.2.11	pg_hba.conf	9
2.2.12	pg_ident.conf	11
2.3	Kontrola i konserwacja	12
2.3.1	Wprowadzenie	12
2.3.2	Zarządzanie przestrzenią i mechanizm współbieżności	12
2.3.3	Optymalizacja i statystyki planisty zapytań	13
2.3.4	Bezpieczeństwo i rygorystyczna kontrola dostępu	13
2.3.5	Strategie zabezpieczania przed utratą danych	13
2.3.6	Podsumowanie	14
2.4	Monitorowanie i diagnostyka	14
2.4.1	Wstęp	14
2.4.2	Sesje i użytkownicy	14
2.4.3	Śledzenie dostępu użytkowników do tabel	15
2.4.4	Błędy dziennika, logi i raporty	15
2.4.5	Monitorowanie na poziomie systemu operacyjnego	15
2.4.6	Monitorowanie serwera	16
2.4.7	Znaczenie monitorowania i diagnostyki w bazach danych	16
2.4.8	Podsumowanie	16
2.4.9	Źródła	16
2.5	Partycjonowanie danych	16
2.6	Bezpieczeństwo Baz Danych	16

2.6.1	1. Model bezpieczeństwa CIA i ochrona infrastruktury	17
2.6.2	2. Ataki na bazy danych i luki w zabezpieczeniach	18
3	Rozdział 3: Projektowanie Bazy Danych: System Zarządzania Biblioteką	20
3.1	Wybór zagadnienia, opis procesów i danych	20
3.1.1	Wybrane zagadnienie:	20
3.1.2	Opis procesów i więzy integralności:	20
3.1.3	Wykaz gromadzonych danych:	20
3.2	Prototyp CSV	20
3.3	Model Konceptualny (Pojęciowy)	21
3.3.1	Zidentyfikowane encje	21
3.3.2	Zdefiniowane atrybuty/własności	21
3.3.3	Opis związków	21
3.3.4	Identyfikacja encji słabych	21
3.3.5	Schemat w notacji Chena	22
3.4	Model logiczny i proces normalizacji	22
3.4.1	Przebieg procesu normalizacji	22
3.4.2	Ostateczna struktura tabel (3NF)	22
3.4.3	Diagram ERD (Model Logiczny)	23
3.5	Model fizyczny bazy danych	23
3.5.1	Model fizyczny dla środowiska SQLite	23
3.5.2	Model fizyczny dla środowiska PostgreSQL	23
4	Rozdział 4: Implementacja fizyczna i zasilanie bazy danych	24
4.1	Definicja fizyczna bazy danych (Zadanie 1)	24
4.1.1	Skrypt dla środowiska PostgreSQL	24
4.1.2	Skrypt dla środowiska SQLite	24
4.2	Mechanizm zasilania bazy danych (Zadanie 2)	25
4.2.1	Formatyzacja danych (Pliki CSV)	25
4.2.2	Implementacja mechanizmu importu	25
4.3	Podsumowanie	26

Autorzy projektu: Paweł Łoćwin, Paweł Łosowski

Wprowadzenie:

Niniejsza dokumentacja stanowi kompletne sprawozdanie z laboratorium przedmiotu Bazy Danych. Projekt obejmuje całościową analizę procesów związanych z administracją i projektowaniem relacyjnych baz danych.

Dokumentacja została podzielona na cztery główne sekcje:

1. **Rozdział 1: Kontrola i konserwacja** – Analiza fundamentalnych procesów administracyjnych niezbędnych do utrzymania wysokiej dostępności i wydajności systemu bazodanowego. Obejmuje VACUUM, ANALYZE i strategie bezpieczeństwa.
2. **Rozdział 2: Badania literaturowe** – Przegląd źródeł akademickich i naukowo-technicznych dotyczących baz danych, ich architektury, optymalizacji i bezpieczeństwa.
3. **Rozdział 3: Projektowanie bazy danych** – Praktyczne modelowanie systemu zarządzania bibliotecznym katalogiem wypożyczeń. Sekcja obejmuje analizę wymagań, model konceptualny (notacja Chen) i schemat logiczny.
4. **Rozdział 4: Implementacja fizyczna i zasilanie bazy** – Realizacja fizycznego schematu bazy danych poprzez skrypty DDL, mechanizmy importu danych oraz automatyzacja zasilania bazy.

ROZDZIAŁ 1: KONTROLA I KONSERWACJA

Autorzy

1. Paweł Łoćwin
2. Paweł Łosowski

1.1 Wprowadzenie

Współczesne systemy relacyjnych baz danych to wysoce skomplikowane środowiska, które wymagają ciągłego i proaktywnego nadzoru, aby zagwarantować wysoką dostępność, niezawodność oraz wydajność. Administracja takimi systemami stanowi złożone zagadnienie wymagające zarówno wiedzy teoretycznej, jak i praktycznego doświadczenia w zarządzaniu infrastrukturą informatyczną.

1.2 Zarządzanie przestrzenią i mechanizm współbieżności

PostgreSQL opiera swoje działanie na zaawansowanej architekturze określonej jako Multi-Version Concurrency Control (MVCC). Technologia ta pozwala na równoległy odczyt i zapis danych bez wzajemnego blokowania się transakcji, co stanowi fundamentalną zaletę w środowiskach wieloużytkownikowych o wysokim natężeniu operacji. Aby zapobiec zjawisku drastycznego puchnięcia tabel oraz degradacji wydajności operacji wejścia i wyjścia, konieczna jest regularna konserwacja na poziomie nośnika danych:

- **VACUUM:** Podstawowy proces konserwacyjny odzyskujący miejsce po martwych krotkach. Oznacza on wolne obszary jako dostępne do ponownego zapisu przez nowe instrukcje, zapobiegając niekontrolowanemu wzrostowi rozmiarów tabel na dysku.
- **VACUUM FULL:** Znacznie bardziej inwazyjna wersja operacji, która fizycznie przebudowuje strukturę całej tabeli i trwale zwraca wolne miejsce do systemu operacyjnego. Operacja ta wymaga jednak zablokowania całej tabeli i powinna być wykonywana w godzinach niskiego obciążenia.
- **AUTOVACUUM:** W nowoczesnych środowiskach produkcyjnych utrzymanie czystości dyskowej powierza się wbudowanemu procesowi pracującemu w tle. Monitoruje on na bieżąco statystyki zmian i automatycznie wyzwała operacje czyszczenia, eliminując potrzebę ręcznej interwencji administratora.

Zaniechanie procesu czyszczenia w systemach o wysokiej rotacji danych może doprowadzić nie tylko do spadku wydajności, ale w skrajnych przypadkach do wyczerpania identyfikatorów transakcji, co prowadzi do całkowitego zamrożenia bazy.

```
-- Przykładowe wymuszenie manualnej konserwacji z analizą statystyk dla tabeli  
VACUUM (VERBOSE, ANALYZE) Wypozyczenia;
```

1.3 Optymalizacja i statystyki planisty zapytań

Kolejnym filarem utrzymania sprawności systemu jest kontrola nad optymalizatorem, zwanym planistą zapytań. Silnik bazy danych przed wykonaniem jakiegokolwiek skomplikowanej kwerendy analizuje dostępne drogi realizacji i wybiera najefektywniejszą z punktu widzenia kosztów obliczeniowych.

Aby planista mógł podejmować optymalne decyzje, musi opierać się na wysoce dokładnych i aktualnych statystykach dotyczących rozkładu danych. Obejmują one między innymi histogramy wartości w kolumnach, liczby odrębnych wartości (distinct count), oraz rozmiary tabel i indeksów.

- Instrukcja **ANALYZE** służy do natychmiastowego odświeżenia tych metadanych. Baza pobiera losową próbkę wierszy i na jej podstawie aktualizuje wewnętrzne tabele systemowe, co pozwala uniknąć błędnych oszacowań planu wykonania.
- Bieżące profilowanie wydajności realizowane jest natomiast przy pomocy instrukcji **EXPLAIN ANALYZE**. Narzędzie to uruchamia zapytanie, po czym zwraca nie tylko wynik, ale również szczegółowy opis planu wykonania wraz z rzeczywistymi czasami i liczbami wierszy na każdym kroku.

```
-- Kontrola planu wykonania zapytania z rzeczywistym pomiarem czasu
EXPLAIN ANALYZE
SELECT c.Imie, c.Nazwisko, k.Tytul
FROM Czytelnicy c
JOIN Wypozyczenia w ON c.ID_Czytelnika = w.ID_Czytelnika
JOIN Ksiazki k ON w.ID_Ksiazki = k.ID_Ksiazki
WHERE w.Data_Zwrotu IS NULL;
```

1.4 Bezpieczeństwo i rygorystyczna kontrola dostępu

Bezpieczeństwo danych stanowi absolutny priorytet każdej administracji systemami informatycznymi. Nowoczesne podejście do ochrony baz danych całkowicie odchodzi od wykorzystywania globalnych kont administratora na rzecz zaawansowanego systemu uprawnień opartego na rolach (RBAC – Role-Based Access Control).

System zabezpieczeń środowiska PostgreSQL pozwala na niezwykle elastyczną gradację uprawnień. Definiuje się dedykowane role systemowe przypisane do konkretnych mikrousług lub modułów aplikacji, z precyzyjnie określonymi prawami dostępu do pojedynczych schematów, tabel, a nawet kolumn.

Kluczowym konceptem jest wdrożenie zasady najmniejszych uprawnień (Principle of Least Privilege). Rola wykorzystywana przez aplikację powinna dysponować prawami pozwalającymi wyłącznie na modyfikację danych niezbędnych do jej funkcjonowania, bez dostępu do danych wrażliwych czy możliwości zmiany struktury bazy.

```
-- Przykład implementacji bezpiecznej polityki kontroli dostępu
CREATE ROLE app_user WITH LOGIN PASSWORD 'TrudneHaslo123';
GRANT CONNECT ON DATABASE biblioteka_db TO app_user;
GRANT USAGE ON SCHEMA public TO app_user;
GRANT SELECT, INSERT ON Wypozyczenia TO app_user;
```

1.5 Strategie zabezpieczania przed utratą danych

Nawet najlepiej zoptymalizowany i zabezpieczony system informatyczny jest narażony na błędy ludzkie lub awarie infrastruktury sprzętowej. Dlatego odpowiednia polityka wykonywania kopii zapasowych stanowi ostateczną linię obrony i jest absolutnie niezbędna dla każdej produkcyjnej instancji bazy danych.

Kopie logiczne polegają na ekstrakcji danych z bazy do postaci czystego skryptu języka SQL lub dedykowanego formatu archiwalnego. Narzędzia realizujące to zadanie gwarantują przenośność danych między platformami i wersjami silnika, jednak proces przywracania jest czasochłonny i może nie przywrócić wszystkich zmian z ostatnich minut przed awarią.

Dlatego w krytycznych środowiskach produkcyjnych stosuje się kopie fizyczne połączone z archiwizacją dzienników wyprzedzających. Dzienniki te rejestrują absolutnie każdą zmianę bajtu na poziomie magazynu, co umożliwia odtworzenie bazy do dowolnego punktu w przeszłości z dokładnością do sekundy.

1.6 Podsumowanie

Prawidłowa kontrola i konserwacja środowiska bazodanowego to wielowymiarowy proces, który nie ma punktu końcowego, lecz trwa przez cały cykl życia oprogramowania. Złożoność nowoczesnych systemów relacyjnych wymaga holistycznego podejścia łączącego automatyzację, monitoring i ciągłe doskonalenie.

Samo zaprojektowanie zgodnego ze sztuką schematu relacyjnego jest zaledwie początkiem pracy. Stabilność aplikacji zależy w równej mierze od rygorystycznego zarządzania fizyczną strukturą danych, optymalizacji wykonywanych operacji oraz wdrażania wielowarstwowych strategii bezpieczeństwa i ochrony przed awarią.

Holistyczne spojrzenie na administrację PostgreSQL, obejmujące zarówno automatyzację procesów porządkujących pamięć, jak i ciągłe monitorowanie planów wykonania zapytań, pozwala na budowanie systemów, które są nie tylko funkcjonalne, ale także odpornie i niezawodne w najbardziej wymagających warunkach produkcyjnych.

BADANIA LITERATUROWE

2.1 Sprzęt dla bazy danych

2.2 Konfiguracja bazy danych PostgreSQL

2.2.1 Wstęp

Rozdział opisuje konfigurację połączenia z bazą danych **PostgreSQL** – dojrzałym, open-source’owym systemem zarządzania relacyjnymi bazami danych (RDBMS), który wyróżnia się pełną zgodnością ze standardem SQL oraz silnym naciskiem na rozszerzalność i niezawodność.

Wymagania:

- dostęp do instancji PostgreSQL,
- zmienne środowiskowe,
- narzędzia projektowe (np. `psql`, `pg_isready`).

2.2.2 Podstawowe parametry

Każde połączenie z PostgreSQL wymaga:

- hosta i portu (domyślnie `localhost` i `5432`),
- nazwy bazy danych,
- użytkownika i hasła.

Dane wrażliwe (np. hasła) przechowuj w **zmiennych środowiskowych** – to oddziela konfigurację od kodu. Pamiętaj: zmienne środowiskowe nie są mechanizmem bezpieczeństwa; w produkcji uzupełnij je o dedykowane zarządzanie sekretami (np. HashiCorp Vault, AWS Secrets Manager).

2.2.3 Lokalizacja i struktura katalogów PostgreSQL

Pliki konfiguracyjne PostgreSQL znajdują się w katalogu danych (PGDATA).

Główne pliki konfiguracyjne:

- `postgresql.conf` – podstawowa konfiguracja serwera (port, pamięć, logi)
- `pg_hba.conf` – kontrola dostępu (kto i skąd może się łączyć)
- `pg_ident.conf` – mapowanie użytkowników systemowych na role PostgreSQL

Domyślne lokalizacje:

Table 1: Domyślne lokalizacje

System	Katalog danych
Linux (RPM/DEB)	<code>/var/lib/pgsql/data/</code> lub <code>/var/lib/postgresql/*/main/</code>
Linux (kompilacja źródłowa)	<code>/usr/local/pgsql/data/</code>
Windows	<code>C:\Program Files\PostgreSQL\<version>\data\</code>
Docker	<code>/var/lib/postgresql/data/</code> (w kontenerze)

Zmiana katalogu danych:

```
# Inicjalizacja nowego katalogu
initdb -D /ścieżka/do/katalogu
```

(continues on next page)

(continued from previous page)

```
# Uruchomienie z niestandardowym PGDATA
pg_ctl -D /ścieżka/do/katalogu start
```

Struktura katalogu danych:

```
$PGDATA/
├── postgresql.conf    # główny plik konfiguracyjny
├── pg_hba.conf        # polityka dostępu
├── pg_ident.conf      # mapowanie tożsamości
├── base/              # rzeczywiste bazy danych
├── global/            # tabele systemowe
├── log/               # logi serwera
└── pg_wal/            # Write-Ahead Log (WAL)
```

Sprawdzenie aktualnej lokalizacji:

```
SHOW data_directory;
SHOW config_file;
SHOW hba_file;
```

2.2.4 Środowiska i profile

Przełączanie środowisk PostgreSQL (dev/test/prod) realizuj przez:

- zmienne środowiskowe (np. PGHOST, PGPORT, PGDATABASE, PGUSER, PGPASSWORD),
- profile konfiguracyjne,
- osobne pliki .env dla każdego środowiska.

2.2.5 Automatyzacja

Zalecenia dla PostgreSQL:

- walidacja konfiguracji przy starcie za pomocą pg_isready,
- skrypty sprawdzające kompletność zmiennych środowiskowych,
- integracja z CI/CD (np. GitHub Actions, GitLab CI).

Dokumentację generuj automatycznie (Sphinx).

2.2.6 Przykłady**PostgreSQL – zmienne środowiskowe (.env)**

```
PGHOST=localhost
PGPORT=5432
PGDATABASE=aplikacja
PGUSER=app_user
PGPASSWORD=${DB_PASSWORD}
```

PostgreSQL – URL połączenia

```
DATABASE_URL=postgresql://${PGUSER}:${PGPASSWORD}@${PGHOST}:${PGPORT}/${PGDATABASE}
```

PostgreSQL – sprawdzenie połączenia

```
pg_isready -h ${PGHOST} -p ${PGPORT} -U ${PGUSER} -d ${PGDATABASE}
```

2.2.7 Najlepsze praktyki

1. Nie przechowuj sekretów w repozytorium.
2. Stosuj `.env.example` zamiast rzeczywistych danych.
3. Używaj standardowych zmiennych środowiskowych PostgreSQL (PG*).
4. Waliduj konfigurację przed uruchomieniem (`pg_isready`).
5. W środowiskach produkcyjnych używaj połączeń SSL/TLS (`sslmode=require`).

2.2.8 Podsumowanie

Poprawna konfiguracja połączenia z PostgreSQL zwiększa bezpieczeństwo i przenośność aplikacji między środowiskami. Standardowe zmienne środowiskowe oraz narzędzia takie jak `pg_isready` ułatwiają automatyzację i walidację.

2.2.9 Szczegółowe omówienie plików konfiguracyjnych

Po omówieniu podstawowych wymagań środowiskowych można przejść do szczegółowej konfiguracji PostgreSQL. W kolejnych sekcjach przedstawiono najważniejsze parametry konfiguracyjne dostępne w plikach:

- `postgresql.conf` – konfiguracja działania serwera,
- `pg_hba.conf` – kontrola dostępu do bazy danych,
- `pg_ident.conf` – mapowanie użytkowników systemowych na role PostgreSQL.

2.2.10 `postgresql.conf`

Plik `postgresql.conf` jest głównym plikiem konfiguracyjnym klastra PostgreSQL. Zawiera ustawienia wpływające na działanie serwera, wydajność, bezpieczeństwo, logowanie oraz replikację.

Lokalizację aktywnego pliku można sprawdzić poleceniem:

```
SHOW config_file;
```

Najważniejsze parametry

Połączenia sieciowe

`listen_addresses`

Określa adresy IP, na których PostgreSQL nasłuchuje połączeń.

Najczęstsze wartości:

- `localhost` – tylko połączenia lokalne,
- `*` – wszystkie interfejsy sieciowe,
- lista adresów, np. `'192.168.1.10,localhost'`.

`port`

Port TCP używany przez PostgreSQL.

Domyślna wartość:

```
port = 5432
```

`max_connections`

Maksymalna liczba jednoczesnych połączeń z bazą danych.

Zbyt wysoka wartość może zwiększać zużycie pamięci.

Pamięć i wydajność

`shared_buffers`

Ilość pamięci RAM używanej przez PostgreSQL do buforowania danych.

Typowe wartości:

- 128MB – środowiska testowe,
- 25% pamięci RAM – środowiska produkcyjne.

work_mem

Ilość pamięci wykorzystywanej przez operacje sortowania i zapytania.

Parametr jest przydzielany dla pojedynczej operacji.

maintenance_work_mem

Pamięć używana podczas operacji administracyjnych, np.:

- VACUUM,
- CREATE INDEX,
- ALTER TABLE.

effective_cache_size

Szacowany rozmiar pamięci podręcznej dostępnej dla PostgreSQL.

Parametr pomaga optymalizatorowi zapytań wybierać odpowiednie plany wykonania.

Logowanie**logging_collector**

Włącza zapisywanie logów do plików.

Dostępne wartości:

- on,
- off.

log_directory

Katalog przechowywania logów.

log_filename

Wzorzec nazw plików logów.

Przykład:

```
log_filename = 'postgresql-%Y-%m-%d.log'
```

log_min_duration_statement

Określa minimalny czas wykonania zapytania (w ms), po którym zostanie ono zapisane w logach.

Przykład:

```
log_min_duration_statement = 1000
```

Wartość 1000 oznacza logowanie zapytań trwających dłużej niż 1 sekundę.

Bezpieczeństwo**password_encryption**

Algorytm szyfrowania haseł użytkowników.

Zalecana wartość:

```
password_encryption = scram-sha-256
```

ssl

Włącza obsługę połączeń SSL/TLS.

Dostępne wartości:

- on,
- off.

ssl_cert_file / ssl_key_file

Ścieżki do certyfikatu i klucza prywatnego SSL.

Replikacja i WAL**wal_level**

Określa poziom szczegółowości Write-Ahead Log (WAL).

Dostępne wartości:

- minimal,

- replica,
- logical.

max_wal_size

Maksymalny rozmiar plików WAL przed wykonaniem checkpointu.

archive_mode

Włącza archiwizację WAL.

archive_command

Polecenie wykonywane podczas archiwizacji plików WAL.

Przykładowa konfiguracja

```
listen_addresses = '*'
port = 5432
max_connections = 200

shared_buffers = 1GB
work_mem = 16MB
maintenance_work_mem = 256MB

logging_collector = on
log_directory = 'log'

ssl = on
password_encryption = scram-sha-256

wal_level = replica
max_wal_size = 2GB
```

Weryfikacja konfiguracji

Po zmianie parametrów konfigurację można przeładować bez restartu serwera:

```
SELECT pg_reload_conf();
```

Niektóre parametry wymagają pełnego restartu PostgreSQL.
Aktualne wartości parametrów można sprawdzić poleceniem:

```
SHOW shared_buffers;
SHOW wal_level;
SHOW max_connections;
```

2.2.11 pg_hba.conf

Plik `pg_hba.conf` (Host-Based Authentication) odpowiada za kontrolę dostępu do serwera PostgreSQL.
Definiuje:

- kto może połączyć się z bazą danych,
- z jakiego adresu IP,
- do jakiej bazy danych,
- przy użyciu jakiej metody uwierzytelniania.

Lokalizację aktywnego pliku można sprawdzić poleceniem:

```
SHOW hba_file;
```

Składnia wpisu

Każdy wpis w `pg_hba.conf` ma następującą strukturę:

TYPE	DATABASE	USER	ADDRESS	METHOD
------	----------	------	---------	--------

Znaczenie pól:

TYPE

Typ połączenia.

Dostępne wartości:

- `local` – połączenia lokalne (Unix socket),
- `host` – połączenia TCP/IP,
- `hostssl` – połączenia TCP/IP z SSL,
- `hostnossl` – połączenia TCP/IP bez SSL.

DATABASE

Nazwa bazy danych, do której użytkownik może się połączyć.

Możliwe wartości:

- konkretna nazwa bazy,
- `all` – wszystkie bazy,
- `sameuser` – baza o nazwie zgodnej z użytkownikiem.

USER

Użytkownik lub rola PostgreSQL.

ADDRESS

Adres IP lub zakres sieci w notacji CIDR.

Przykłady:

- `127.0.0.1/32`,
- `192.168.1.0/24`,
- `0.0.0.0/0` – wszystkie adresy.

METHOD

Metoda uwierzytelniania użytkownika.

Najczęściej używane metody uwierzytelniania

trust

Zezwala na połączenie bez uwierzytelniania.

Zalecane wyłącznie w środowiskach testowych.

reject

Odrzuca połączenie.

md5

Uwierzytelnianie przy użyciu hasła MD5.

Metoda starsza i mniej bezpieczna niż SCRAM.

scram-sha-256

Nowoczesna metoda uwierzytelniania oparta na SCRAM.

Zalecana dla nowych instalacji PostgreSQL.

peer

Uwierzytelnianie na podstawie użytkownika systemowego.

Najczęściej używane dla lokalnych połączeń Linux/Unix.

ident

Mapowanie użytkownika systemowego przy użyciu `pg_ident.conf`.

Przykładowa konfiguracja

```
# Połączenia lokalne
local    all                                postgres                                peer
```

(continues on next page)

(continued from previous page)

```
# Połączenia localhost IPv4
host    all                all                127.0.0.1/32      scram-sha-256

# Połączenia localhost IPv6
host    all                all                ::1/128           scram-sha-256

# Sieć lokalna
host    aplikacja          app_user          192.168.1.0/24    scram-sha-256
```

Najlepsze praktyki

1. Używaj `scram-sha-256` zamiast `md5`.
2. Ograniczaj dostęp do konkretnych adresów IP.
3. Nie używaj `trust` w środowiskach produkcyjnych.
4. Stosuj `hostssl` dla połączeń zdalnych.
5. Po zmianie konfiguracji wykonuj reload konfiguracji:

```
SELECT pg_reload_conf();
```

2.2.12 pg_ident.conf

Plik `pg_ident.conf` umożliwia mapowanie użytkowników systemowych na role PostgreSQL.

Najczęściej jest używany jest razem z metodami uwierzytelniania:

- `peer`,
- `ident`.

Lokalizację aktywnego pliku można sprawdzić poleceniem:

```
SHOW ident_file;
```

Zastosowanie

Mechanizm mapowania pozwala określić, który użytkownik systemowy może zostać powiązany z konkretną rolą PostgreSQL.

Jest to szczególnie przydatne:

- w środowiskach Linux/Unix,
- przy automatyzacji administracyjnej,
- dla lokalnych połączeń bez użycia haseł.

Składnia wpisu

MAPNAME	SYSTEM-USERNAME	PG-USERNAME
---------	-----------------	-------------

Znaczenie pól:

MAPNAME

Nazwa mapowania wykorzystywana w `pg_hba.conf`.

SYSTEM-USERNAME

Użytkownik systemu operacyjnego.

PG-USERNAME

Rola PostgreSQL, na którą użytkownik ma zostać zmapowany.

Przykładowa konfiguracja

```
# MAPNAME      SYSTEM-USERNAME  PG-USERNAME
admin_map      postgres          postgres
admin_map      deploy           db_admin
app_map        appuser          aplikacja
```

Przykład użycia w `pg_hba.conf`:

```
local  all  all  peer map=admin_map
```

W powyższym przykładzie użytkownicy systemowi zdefiniowani w `admin_map` mogą logować się jako odpowiadające im role PostgreSQL.

Najlepsze praktyki

1. Ograniczaj mapowania wyłącznie do wymaganych użytkowników.
2. Nie mapuj użytkowników systemowych na role superuser bez uzasadnienia.
3. Stosuj osobne role administracyjne i aplikacyjne.
4. Dokumentuj wszystkie mapowania użytkowników.
5. Regularnie przeglądaj konfigurację dostępu.

Opracowanie

Autor: Michał Kraus **Przedmiot:** Bazy danych **Data:** maj 2026

2.3 Kontrola i konserwacja

Autorzy

1. Paweł Łoćwin
2. Paweł Łosowski

2.3.1 Wprowadzenie

Współczesne systemy relacyjnych baz danych to wysoce skomplikowane środowiska, które wymagają ciągłego i proaktywnego nadzoru, aby zagwarantować wysoką dostępność, niezawodność oraz wydajność. Administracja takimi systemami stanowi złożone zagadnienie wymagające zarówno wiedzy teoretycznej, jak i praktycznego doświadczenia w zarządzaniu infrastrukturą informatyczną.

2.3.2 Zarządzanie przestrzenią i mechanizm współbieżności

PostgreSQL opiera swoje działanie na zaawansowanej architekturze określanej jako Multi-Version Concurrency Control (MVCC). Technologia ta pozwala na równoległy odczyt i zapis danych bez wzajemnego blokowania się transakcji, co stanowi fundamentalną zaletę w środowiskach wieloużytkownikowych o wysokim natężeniu operacji. Aby zapobiec zjawisku drastycznego puchnięcia tabel oraz degradacji wydajności operacji wejścia i wyjścia, konieczna jest regularna konserwacja na poziomie nośnika danych:

- **VACUUM:** Podstawowy proces konserwacyjny odzyskujący miejsce po martwych krotkach. Oznacza on wolne obszary jako dostępne do ponownego zapisu przez nowe instrukcje, zapobiegając niekontrolowanemu wzrostowi rozmiarów tabel na dysku.
- **VACUUM FULL:** Znacznie bardziej inwazyjna wersja operacji, która fizycznie przebudowuje strukturę całej tabeli i trwale zwraca wolne miejsce do systemu operacyjnego. Operacja ta wymaga jednak zablokowania całej tabeli i powinna być wykonywana w godzinach niskiego obciążenia.
- **AUTOVACUUM:** W nowoczesnych środowiskach produkcyjnych utrzymanie czystości dyskowej powierza się wbudowanemu procesowi pracującemu w tle. Monitoruje on na bieżąco statystyki zmian i automatycznie wyzwała operacje czyszczenia, eliminując potrzebę ręcznej interwencji administratora.

Zaniechanie procesu czyszczenia w systemach o wysokiej rotacji danych może doprowadzić nie tylko do spadku wydajności, ale w skrajnych przypadkach do wyczerpania identyfikatorów transakcji, co prowadzi do całkowitego zamrożenia bazy.

```
-- Przykładowe wymuszenie manualnej konserwacji z analizą statystyk dla tabeli
VACUUM (VERBOSE, ANALYZE) Wypozyczenia;
```

2.3.3 Optymalizacja i statystyki planisty zapytań

Kolejnym filarem utrzymania sprawności systemu jest kontrola nad optymalizatorem, zwanym planistą zapytań. Silnik bazy danych przed wykonaniem jakiegokolwiek skomplikowanej kwerendy analizuje dostępne drogi realizacji i wybiera najefektywniejszą z punktu widzenia kosztów obliczeniowych.

Aby planista mógł podejmować optymalne decyzje, musi opierać się na wysoce dokładnych i aktualnych statystykach dotyczących rozkładu danych. Obejmują one między innymi histogramy wartości w kolumnach, liczby odrębnych wartości (distinct count), oraz rozmiary tabel i indeksów.

- Instrukcja **ANALYZE** służy do natychmiastowego odświeżenia tych metadanych. Baza pobiera losową próbkę wierszy i na jej podstawie aktualizuje wewnętrzne tabele systemowe, co pozwala uniknąć błędnych oszacowań planu wykonania.
- Bieżące profilowanie wydajności realizowane jest natomiast przy pomocy instrukcji **EXPLAIN ANALYZE**. Narzędzie to uruchamia zapytanie, po czym zwraca nie tylko wynik, ale również szczegółowy opis planu wykonania wraz z rzeczywistymi czasami i liczbami wierszy na każdym kroku.

```
-- Kontrola planu wykonania zapytania z rzeczywistym pomiarem czasu
EXPLAIN ANALYZE
SELECT c.Imie, c.Nazwisko, k.Tytul
FROM Czytelnicy c
JOIN Wypozyczenia w ON c.ID_Czytelnika = w.ID_Czytelnika
JOIN Ksiazki k ON w.ID_Ksiazki = k.ID_Ksiazki
WHERE w.Data_Zwrotu IS NULL;
```

2.3.4 Bezpieczeństwo i rygorystyczna kontrola dostępu

Bezpieczeństwo danych stanowi absolutny priorytet każdej administracji systemami informatycznymi. Nowoczesne podejście do ochrony baz danych całkowicie odchodzi od wykorzystywania globalnych kont administratora na rzecz zaawansowanego systemu uprawnień opartego na rolach (RBAC – Role-Based Access Control).

System zabezpieczeń środowiska PostgreSQL pozwala na niezwykle elastyczną gradację uprawnień. Definiuje się dedykowane role systemowe przypisane do konkretnych mikrousług lub modułów aplikacji, z precyzyjnie określonymi prawami dostępu do pojedynczych schematów, tabel, a nawet kolumn.

Kluczowym konceptem jest wdrożenie zasady najmniejszych uprawnień (Principle of Least Privilege). Rola wykorzystywana przez aplikację powinna dysponować prawami pozwalającymi wyłącznie na modyfikację danych niezbędnych do jej funkcjonowania, bez dostępu do danych wrażliwych czy możliwości zmiany struktury bazy.

```
-- Przykład implementacji bezpiecznej polityki kontroli dostępu
CREATE ROLE app_user WITH LOGIN PASSWORD 'TrudneHaslo123';
GRANT CONNECT ON DATABASE biblioteka_db TO app_user;
GRANT USAGE ON SCHEMA public TO app_user;
GRANT SELECT, INSERT ON Wypozyczenia TO app_user;
```

2.3.5 Strategie zabezpieczania przed utratą danych

Nawet najlepiej zoptymalizowany i zabezpieczony system informatyczny jest narażony na błędy ludzkie lub awarie infrastruktury sprzętowej. Dlatego odpowiednia polityka wykonywania kopii zapasowych stanowi ostateczną linię obrony i jest absolutnie niezbędna dla każdej produkcyjnej instancji bazy danych.

Kopie logiczne polegają na ekstrakcji danych z bazy do postaci czystego skryptu języka SQL lub dedykowanego formatu archiwalnego. Narzędzia realizujące to zadanie gwarantują przenośność danych między platformami i wersjami silnika, jednak proces przywracania jest czasochłonny i może nie przywrócić wszystkich zmian z ostatnich minut przed awarią.

Dlatego w krytycznych środowiskach produkcyjnych stosuje się kopie fizyczne połączone z archiwizacją dzienników wyprzedzających. Dzienniki te rejestrują absolutnie każdą zmianę bajtu na poziomie magazynu, co umożliwia odtworzenie bazy do dowolnego punktu w przeszłości z dokładnością do sekundy.

2.3.6 Podsumowanie

Prawidłowa kontrola i konserwacja środowiska bazodanowego to wielowymiarowy proces, który nie ma punktu końcowego, lecz trwa przez cały cykl życia oprogramowania. Złożoność nowoczesnych systemów relacyjnych wymaga holistycznego podejścia łączącego automatyzację, monitoring i ciągłe doskonalenie.

Samo zaprojektowanie zgodnego ze sztuką schematu relacyjnego jest zaledwie początkiem pracy. Stabilność aplikacji zależy w równej mierze od rygorystycznego zarządzania fizyczną strukturą danych, optymalizacji wykonywanych operacji oraz wdrażania wielowarstwowych strategii bezpieczeństwa i ochrony przed awarią.

Holistyczne spojrzenie na administrację PostgreSQL, obejmujące zarówno automatyzację procesów porządkujących pamięć, jak i ciągłe monitorowanie planów wykonania zapytań, pozwala na budowanie systemów, które są nie tylko funkcjonalne, ale także odporne i niezawodne w najbardziej wymagających warunkach produkcyjnych.

2.4 Monitorowanie i diagnostyka

2.4.1 Wstęp

Monitorowanie i diagnostyka baz danych obejmują zestaw działań, które pozwalają administratorowi lub zespołowi utrzymania sprawdzić, czy system działa poprawnie, bezpiecznie i wydajnie. W praktyce nie chodzi jedynie o obserwowanie pojedynczych parametrów serwera, ale o łączenie informacji pochodzących z wielu warstw: samej bazy danych, systemu operacyjnego, aplikacji oraz narzędzi zewnętrznych. Dzięki temu można szybciej wykrywać przeciążenia, błędy konfiguracji, problemy z zapytaniami SQL, nieprawidłowe uprawnienia oraz awarie sprzętowe lub sieciowe.

W systemach bazodanowych szczególnie ważne jest to, że wiele problemów nie jest od razu widocznych dla użytkownika końcowego. Przykładowo zapytania mogą stopniowo wykonywać się coraz wolniej, liczba aktywnych sesji może rosnąć, a logi mogą zawierać powtarzające się ostrzeżenia. Bez monitorowania takie sygnały są łatwe do przeoczenia. Diagnostyka pozwala natomiast przejść od samego wykrycia problemu do ustalenia jego przyczyny, np. przez analizę sesji, blokad, planów zapytań, dzienników błędów lub zużycia zasobów systemowych.

2.4.2 Sesje i użytkownicy

Jednym z podstawowych obszarów monitorowania bazy danych jest obserwowanie aktywnych sesji i użytkowników. Sesja oznacza połączenie użytkownika lub aplikacji z serwerem bazy danych. W ramach sesji wykonywane są zapytania, transakcje oraz operacje administracyjne. Analiza sesji pozwala sprawdzić między innymi, kto jest aktualnie połączony z bazą, z jakiego hosta pochodzi połączenie, jakie zapytanie jest wykonywane oraz czy sesja jest aktywna, bezczynna albo zablokowana.

W wielu systemach zarządzania bazami danych dostępne są specjalne widoki lub narzędzia administracyjne służące do obserwowania bieżącej aktywności. Na przykład PostgreSQL udostępnia mechanizmy monitorowania aktywności bazy danych oraz widoki statystyczne, takie jak `pg_stat_activity`, które pozwalają sprawdzać informacje o procesach serwera i wykonywanych zapytaniach¹. W MySQL podobną rolę może pełnić Performance Schema, które gromadzi dane diagnostyczne o pracy serwera i może być wykorzystywane do analizy wydajności². Monitorowanie sesji jest istotne zarówno z punktu widzenia wydajności, jak i bezpieczeństwa. Zbyt duża liczba jednoczesnych połączeń może prowadzić do przeciążenia serwera, a długo działające transakcje mogą blokować

¹ PostgreSQL Documentation, *Monitoring Database Activity* oraz *The Cumulative Statistics System*, <https://www.postgresql.org/docs/current/monitoring.html>

² MySQL Documentation, *Using the Performance Schema to Diagnose Problems*, <https://dev.mysql.com/doc/refman/8.2/en/performance-schema-examples.html>

inne operacje. Z kolei nietypowe połączenia, np. z nieznanych adresów lub wykonywane poza normalnymi godzinami pracy, mogą wskazywać na błąd konfiguracji albo próbę nieautoryzowanego dostępu.

2.4.3 Śledzenie dostępu użytkowników do tabel

Kolejnym ważnym elementem jest kontrola tego, którzy użytkownicy uzyskują dostęp do określonych tabel i jakie operacje wykonują. W bazach danych przechowywane są często dane wrażliwe, dlatego sama informacja o tym, że użytkownik posiada konto w systemie, nie jest wystarczająca. Należy również wiedzieć, czy jego działania są zgodne z nadanymi uprawnieniami oraz z zasadami bezpieczeństwa obowiązującymi w organizacji.

Śledzenie dostępu może obejmować operacje odczytu danych, modyfikacji rekordów, usuwania danych, tworzenia nowych obiektów oraz zmian w uprawnieniach. W praktyce realizuje się to za pomocą mechanizmów audytu, dzienników zapytań, wyzwalaczy, narzędzi klasy Database Activity Monitoring lub funkcji wbudowanych w konkretny system bazodanowy. Przykładowo Oracle Database udostępnia mechanizmy audytu, a nowsze wersje zalecają korzystanie z unified auditing zamiast starszego audytu tradycyjnego³.

Takie śledzenie jest potrzebne nie tylko przy wykrywaniu incydentów bezpieczeństwa. Pomaga również w spełnianiu wymagań formalnych, analizie błędów użytkowników oraz odtwarzaniu przebiegu zdarzeń po awarii. Przykładowo, jeśli w tabeli pojawiły się nieprawidłowe dane, historia dostępu może pomóc ustalić, kiedy doszło do zmiany i która aplikacja lub który użytkownik ją wykonał.

2.4.4 Błędy dziennika, logi i raporty

Dzienniki zdarzeń, czyli logi, są jednym z najważniejszych źródeł informacji diagnostycznych. Serwer bazy danych może zapisywać w nich komunikaty o błędach, ostrzeżeniach, nieudanych logowaniach, problemach z zapytaniami, restartach, zmianach konfiguracji lub przekroczeniu określonych progów wydajnościowych. Analiza logów pozwala wykrywać problemy, które nie zawsze są widoczne w aplikacji klienckiej.

W zależności od systemu bazodanowego logi mogą mieć różną postać. PostgreSQL obsługuje różne metody logowania komunikatów serwera, między innymi stderr, csvlog, jsonlog oraz syslog; w systemie Windows możliwe jest także użycie dziennika zdarzeń⁴. MySQL udostępnia między innymi general query log, który może rejestrować połączenia i zapytania otrzymywane przez serwer, oraz slow query log, używany do identyfikowania zapytań wykonujących się zbyt długo⁵.

Raporty diagnostyczne są rozwinięciem surowych logów. Zamiast ręcznie analizować tysiące wpisów, administrator może korzystać z raportów podsumowujących liczbę błędów, najwolniejsze zapytania, częstotliwość nieudanych logowań lub zmiany obciążenia w czasie. Raporty są szczególnie przydatne w pracy zespołowej, ponieważ pozwalają dokumentować problemy i porównywać stan systemu przed oraz po wprowadzeniu zmian optymalizacyjnych.

2.4.5 Monitorowanie na poziomie systemu operacyjnego

Baza danych działa na konkretnym systemie operacyjnym i korzysta z jego zasobów. Dlatego diagnostyka nie może ograniczać się wyłącznie do samego silnika bazy danych. Wydajność zapytań może zależeć od procesora, pamięci RAM, operacji wejścia/wyjścia na dysku, sieci, liczby procesów oraz ogólnego obciążenia systemu.

W systemach Linux często wykorzystuje się narzędzia takie jak iostat, htop oraz vmstat. Narzędzie iostat pozwala obserwować wykorzystanie procesora i urządzeń wejścia/wyjścia, co jest szczególnie ważne przy bazach danych intensywnie korzystających z dysku. htop umożliwia wygodne śledzenie procesów i zużycia zasobów w czasie rzeczywistym. vmstat pokazuje między innymi informacje o pamięci, procesach, wymianie danych z dyskiem oraz obciążeniu procesora.

W systemach Microsoft Windows podobną rolę pełnią Menedżer zadań oraz Monitor wydajności. Pozwalają one obserwować zużycie procesora, pamięci, dysków, sieci oraz usług działających w tle. W środowiskach produkcyjnych takie dane są często zbierane automatycznie i zestawiane z metrykami pochodzącymi z bazy danych. Dzięki temu można odróżnić problem wynikający z nieoptymalnego zapytania SQL od problemu spowodowanego np. przeciążeniem dysku lub brakiem pamięci.

³ Oracle Documentation, *AUDIT_TRAIL* oraz informacje o unified auditing, https://docs.oracle.com/en/database/oracle/oracle-database/21/refrn/AUDIT_TRAIL.html

⁴ PostgreSQL Documentation, *Error Reporting and Logging*, <https://www.postgresql.org/docs/current/runtime-config-logging.html>

⁵ MySQL Documentation, *The General Query Log* oraz *The Slow Query Log*, <https://dev.mysql.com/doc/en/query-log.html>

2.4.6 Monitorowanie serwera

Ostatnim poziomem jest monitorowanie całego serwera lub infrastruktury, w której działa baza danych. Obejmuje ono dostępność usług, czas odpowiedzi, zużycie zasobów, stan dysków, wykorzystanie sieci, działanie kopii zapasowych oraz wysyłanie powiadomień w przypadku awarii. W tym celu stosuje się narzędzia takie jak Nagios, Grafana, Prometheus, Zabbix lub inne systemy monitoringu.

Nagios jest przykładem narzędzia używanego do monitorowania usług i hostów, sprawdzania ich stanu oraz informowania administratorów o problemach⁶. Grafana z kolei służy przede wszystkim do wizualizacji danych monitorujących w formie dashboardów. Panele Grafany mogą prezentować metryki w postaci wykresów, tabel i innych wizualizacji, a moduł alertów pozwala reagować na określone zdarzenia lub przekroczenie progów⁷.

Monitorowanie serwera jest szczególnie ważne w środowiskach, w których baza danych stanowi element większego systemu. Awaria bazy może wynikać nie tylko z błędu samego silnika, ale również z problemu sieciowego, pełnionego dysku, braku pamięci, błędnej konfiguracji systemu lub nieudanego wdrożenia aplikacji. Centralne monitorowanie pozwala zebrać te informacje w jednym miejscu i szybciej wskazać źródło problemu.

2.4.7 Znaczenie monitorowania i diagnostyki w bazach danych

Monitorowanie i diagnostyka są ważne, ponieważ wpływają na trzy kluczowe obszary: dostępność, wydajność i bezpieczeństwo danych. Dostępność oznacza, że baza danych jest osiągalna dla użytkowników i aplikacji. Wydajność oznacza, że zapytania wykonują się w akceptowalnym czasie. Bezpieczeństwo oznacza natomiast, że dostęp do danych jest kontrolowany, a podejrzane działania mogą zostać wykryte i przeanalizowane.

W dobrze utrzymanym środowisku bazodanowym monitorowanie powinno działać stale, a nie dopiero wtedy, gdy pojawi się awaria. Regularne obserwowanie sesji, logów, dostępu do tabel, parametrów systemu operacyjnego i stanu serwera pozwala wykrywać problemy na wczesnym etapie. Diagnostyka pozwala natomiast ustalić przyczyny tych problemów i zaplanować odpowiednie działania, takie jak optymalizacja zapytań, zmiana konfiguracji, zwiększenie zasobów, poprawa uprawnień lub wdrożenie dodatkowych zabezpieczeń.

2.4.8 Podsumowanie

Monitorowanie i diagnostyka baz danych tworzą wielowarstwowy proces kontroli działania systemu. Na poziomie bazy danych analizuje się sesje, użytkowników, zapytania, blokady, dostęp do tabel oraz logi. Na poziomie systemu operacyjnego obserwuje się zużycie procesora, pamięci, dysku i sieci. Na poziomie serwera wykorzystuje się narzędzia monitorujące, które zbierają metryki, tworzą wykresy oraz wysyłają alerty. Dopiero połączenie tych perspektyw daje pełny obraz stanu systemu i pozwala skutecznie reagować na błędy, spadki wydajności oraz potencjalne incydenty bezpieczeństwa.

2.4.9 Źródła

Tego typu jakis tekst

2.5 Partycjonowanie danych

2.6 Bezpieczeństwo Baz Danych

Dział 7 przeglądu literaturowego poświęcony architekturze bezpieczeństwa, mechanizmom ochrony oraz analizie podatności w nowoczesnych systemach zarządzania bazami danych.

⁶ Nagios Open Source Documentation, <https://www.nagios.org/documentation/>

⁷ Grafana Documentation, *Dashboards overview* oraz *Grafana Alerting*, <https://grafana.com/docs/grafana/latest/fundamentals/dashboards-overview/>

2.6.1 1. Model bezpieczeństwa CIA i ochrona infrastruktury

Rozważając bezpieczeństwo systemów bazodanowych, należy spojrzeć na problem warstwowo. Najniższą, a zarazem najbardziej fundamentalną warstwą jest ochrona samej infrastruktury sprzętowej i sieciowej, na której operuje silnik bazy danych (DBMS). Zanim przejdziemy do szczegółowych konfiguracji silnika, konieczne jest zdefiniowanie nadrzędnych celów bezpieczeństwa oraz zapewnienie izolacji środowiska uruchomieniowego.

Triada CIA w środowisku bazodanowym

Model **CIA** (ang. *Confidentiality, Integrity, Availability*) to klasyczny paradygmat bezpieczeństwa informacji. W kontekście relacyjnych i nierelacyjnych baz danych każdy z tych elementów realizowany jest przez specyficzne mechanizmy:

- **Poufność (Confidentiality):** Gwarantuje, że dane są dostępne wyłącznie dla autoryzowanych podmiotów. W bazach danych osiąga się to poprzez ścisłą kontrolę dostępu (np. RBAC), szyfrowanie danych w spoczynku (TDE - *Transparent Data Encryption*), maskowanie danych wrażliwych oraz szyfrowanie połączeń sieciowych (TLS).
- **Integralność (Integrity):** Zapewnia spójność i poprawność danych, chroniąc je przed nieautoryzowaną modyfikacją. Silniki bazodanowe realizują ten postulat natywnie poprzez właściwości ACID (szczególnie *Consistency*), więzy integralności (klucze obce, ograniczenia *CHECK*), a z punktu widzenia bezpieczeństwa – poprzez audytowanie DML (Data Manipulation Language) i mechanizmy wykrywania anomalii.
- **Dostępność (Availability):** Oznacza pewność, że system odpowie na żądanie w akceptowalnym czasie. Zagrożeniem dla dostępności są awarie sprzętowe oraz ataki DoS. Obroną na poziomie bazy danych są klastry wysokiej dostępności (HA), replikacja (np. *Master-Slave, Multi-Master*), partycjonowanie oraz odpowiednio zaprojektowane mechanizmy Disaster Recovery.

Izolacja sieciowa i architektura chmurowa (VPC)

Nawet najlepiej skonfigurowany system uprawnień bazodanowych traci na znaczeniu, jeśli instancja wystawiona jest bezpośrednio do publicznej sieci Internet. Standardem inżynierskim w nowoczesnych wdrożeniach (zarówno on-premise, jak i cloud) jest głęboka separacja sieciowa.

W środowiskach chmurowych (np. AWS, Azure, GCP) bazy danych umieszcza się w dedykowanych chmurach prywatnych (VPC - *Virtual Private Cloud*). Dobrą praktyką jest wdrożenie architektury wielowarstwowej:

1. **Warstwa publiczna (Public Subnet):** Zawiera load balancery i serwery webowe (np. Nginx), do których dostęp ma użytkownik końcowy.
2. **Warstwa aplikacji (Private Subnet):** Środowisko uruchomieniowe backendu (np. Node.js, Spring Boot).
3. **Warstwa danych (Database Subnet):** Najgłębiej ukryta podsieć prywatna bez routingu do Internetu (brak *Internet Gateway*). Dostęp do niej ma wyłącznie warstwa aplikacji poprzez ściśle określone reguły *Security Groups* lub *Network ACLs* (dopuszczające ruch tylko na konkretnym porcie, np. 5432 dla PostgreSQL, i tylko ze znanych adresów IP warstwy aplikacyjnej).

Takie podejście drastycznie zmniejsza powierzchnię ataku (ang. *attack surface*), eliminując ryzyko bezpośrednich ataków typu *brute-force* na porty bazy danych z zewnątrz.

Konteneryzacja a bezpieczeństwo bazy

Uruchamianie baz danych w kontenerach (np. Docker, Kubernetes) stało się powszechne, jednak niesie ze sobą specyficzne wektory zagrożeń. Kontenery dzielą jądro (kernel) z systemem hosta, co oznacza, że kompromitacja bazy danych może prowadzić do ucieczki z kontenera (ang. *container breakout*).

Aby zminimalizować to ryzyko, stosuje się następujące praktyki:

- **Brak uprawnień roota:** Procesy DBMS (np. *postgres* lub *mysqld*) nigdy nie powinny działać jako użytkownik *root* wewnątrz kontenera. Standardowe obrazy często wymagają jawnego zdefiniowania dyrektywy *USER* w pliku *Dockerfile*.
- **Ograniczenia zasobów (Cgroups):** Podatność silnika bazy na ataki typu *Denial of Service* można złagodzić na poziomie infrastruktury, nakładając twarde limity na zużycie CPU i pamięci RAM przez dany kontener. Zapobiega to sytuacji, w której przeciążona baza kładzie cały serwer hosta (Resource Exhaustion).
- **Read-Only Root Filesystem:** Kontener bazy danych powinien mieć zamontowany system plików w trybie tylko do odczytu, z wyjątkiem ściśle zdefiniowanych wolumenów (ang. *volumes*) przeznaczonych na

pliki danych (np. /var/lib/postgresql/data). Uniemożliwia to atakującemu wstrzyknięcie złośliwych skryptów do katalogów systemowych po udanym wykorzystaniu luki aplikacyjnej.

2.6.2 2. Ataki na bazy danych i luki w zabezpieczeniach

Ponieważ bazy danych przechowują najbardziej krytyczne informacje organizacji, stanowią główny cel cyberataków. Według zestawień takich jak OWASP Top 10, podatności związane ze wstrzykiwaniem kodu (Injection) oraz błędną konfiguracją od lat utrzymują się w czołówce zagrożeń. Zrozumienie mechaniki tych ataków jest kluczowe do zaprojektowania skutecznych mechanizmów obronnych w warstwie aplikacyjnej i bazodanowej.

Wstrzykiwanie kodu SQL (SQL Injection)

SQL Injection (SQLi) to podatność polegająca na możliwości modyfikacji zapytania SQL poprzez nieodpowiednio zwalidowane dane wejściowe. Pozwala to atakującemu na wykonanie nieautoryzowanych operacji na bazie danych. Klasyczny przykład podatności (język PHP):

```
$username = $_POST['username'];
$query = "SELECT * FROM users WHERE username = '$username'";
$db->execute($query);
```

Jeśli atakujący jako parametr username przekaże ciąg ' OR '1'='1, zapytanie przyjmie postać:

```
SELECT * FROM users WHERE username = '' OR '1'='1';
```

Ponieważ warunek '1'='1' jest zawsze prawdziwy, silnik bazy danych zwróci wszystkie rekordy z tabeli users, omijając mechanizm uwierzytelniania.

Rodzaje ataków SQLi:

1. **In-Band SQLi (Classic):** Atakujący używa tego samego kanału komunikacyjnego do przeprowadzenia ataku i zebrania wyników. Najpopularniejsze to *Error-based SQLi* (wymuszanie błędów silnika w celu wycieku metadanych, np. struktury tabel) oraz *Union-based SQLi* (użycie operatora UNION do dołączenia złośliwych zapytań do oryginalnego).
2. **Inferential (Blind) SQLi:** Występuje, gdy aplikacja nie zwraca komunikatów o błędach ani bezpośrednich wyników zapytań. Atakujący musi wnioskować o strukturze danych na podstawie zachowania aplikacji. * **Boolean-based:** Wysyłanie zapytań warunkowych i obserwowanie, czy strona ładuje się inaczej. * **Time-based:** Zmuszanie bazy do wstrzymania działania (np. funkcjami pg_sleep() w PostgreSQL lub WAITFOR DELAY w SQL Server), aby potwierdzić obecność podatności.
3. **Out-of-Band SQLi:** Wykorzystywane rzadziej, opiera się na wymuszeniu przez silnik bazy danych żądania HTTP lub DNS do zewnętrznego serwera kontrolowanego przez atakującego.

Mitygacja: Podstawową i najskuteczniejszą metodą obrony przed SQLi jest korzystanie z **zapytań parametryzowanych (Prepared Statements)** lub nowoczesnych systemów ORM (Object-Relational Mapping), które oddzielają składnię zapytań od danych wejściowych.

Ataki NoSQL Injection

Wraz ze wzrostem popularności nierelacyjnych baz danych (np. MongoDB, CouchDB), pojawił się mit o ich całkowitej odporności na wstrzykiwanie kodu. Choć nie używają one tradycyjnego dialektu SQL, aplikacje z nich korzystające nadal mogą być podatne na modyfikację logiki zapytań.

Ataki te często bazują na wstrzykiwaniu operatorów bazy danych do zapytań, które aplikacja interpretuje jako obiekty JSON.

Przykład dla MongoDB: Aplikacja Node.js oczekuje poświadczeń przekazanych w formacie JSON:

```
db.collection('users').find({
  username: req.body.username,
  password: req.body.password
});
```

Atakujący wysyła zmodyfikowany payload z użyciem operatora logicznego \$ne (Not Equal):


```
{
  "username": {"$ne": null},
  "password": {"$ne": null}
}
```

W rezultacie silnik MongoDB wykonuje zapytanie szukające użytkownika, którego login i hasło *nie są nullem*. Zapytanie zwraca pierwszy pasujący rekord (często administratora), umożliwiając logowanie bez znajomości hasła.

Przepełnienie bufora (Buffer Overflow) w DBMS

Silniki baz danych (np. MySQL, PostgreSQL, Oracle) to skomplikowane aplikacje napisane najczęściej w językach C lub C++, co oznacza, że są potencjalnie narażone na błędy zarządzania pamięcią.

Buffer Overflow w kontekście baz danych występuje, gdy atakujący przesyła nieproporcjonalnie duży ciąg danych do bufora o stałej wielkości (np. w parametrze funkcji wbudowanej, mechanizmie autoryzacyjnym lub procedurze składowanej). Gdy dane wykraczają poza zaalokowany obszar pamięci, mogą nadpisać sąsiadujące obszary, w tym wskaźnik powrotu instrukcji (Instruction Pointer).

Może to prowadzić do: 1. **Awarji silnika bazy danych (Crash)** – przerywając ciągłość działania. 2. **Wykonania dowolnego kodu (RCE - Remote Code Execution)** – uruchomienia shellcode'u na serwerze z uprawnieniami procesu bazy danych, co stanowi bezpośrednie wejście do kompromitacji hosta (powiązane z ryzykiem opisanym w rozdziale o konteneryzacji).

Odmowa Usługi (DoS) na poziomie bazy danych

Ataki Denial of Service na bazy danych różnią się od klasycznych ataków sieciowych. Mają one charakter aplikacyjny i celują w wyczerpanie zasobów sprzętowych serwera (CPU, RAM, I/O) lub limitów samego oprogramowania.

- **Wyczerpanie puli połączeń (Connection Exhaustion):** Każdy DBMS ma zdefiniowany limit maksymalnej liczby jednoczesnych połączeń (np. parametr `max_connections` w PostgreSQL). Atakujący, często wykorzystując luki w warstwie aplikacji, może zainicjować tysiące zawieszonych lub bardzo powolnych sesji bazy, uniemożliwiając logowanie legalnym użytkownikom.
- **Asymetryczne obciążenie (Query-based DoS):** Atakujący wstrzykuje i uruchamia zapytania, które są syntaktycznie poprawne, ale drastycznie nieoptymalne kosztowo. Przykładem może być wymuszenie iloczynu kartezjańskiego (CROSS JOIN) na bardzo dużych tabelach bez odpowiednich indeksów, co może zablokować procesor serwera na kilka minut i doprowadzić do blokady całej aplikacji (Deadlock/Timeouts).

Automatyzacja ataków: Narzędzia audytowe

W procesach testów penetracyjnych baz danych, analitycy bezpieczeństwa korzystają ze zautomatyzowanych frameworków. Najpopularniejszym z nich jest **sqlmap** – potężne narzędzie open-source automatyzujące proces wykrywania podatności SQLi i przejmowania kontroli nad serwerami bazodanowymi.

W typowym scenariuszu audytu, uruchomienie polecenia:

```
sqlmap -u "http://podatna-aplikacja.local/item.php?id=1" --dbs
```

pozwoli narzędziu na wykrycie typu wstrzyknięcia (np. czasowe lub błędowe), zidentyfikowanie używanego silnika DBMS oraz – w razie sukcesu – wyliczenie wszystkich baz danych dostępnych na serwerze (enumeracja instancji).

ROZDZIAŁ 3: PROJEKTOWANIE BAZY DANYCH: SYSTEM ZARZĄDZANIA BIBLIOTEKĄ

Autorzy

1. Paweł Łoćwin
2. Paweł Łosowski

3.1 Wybór zagadnienia, opis procesów i danych

3.1.1 Wybrane zagadnienie:

System zarządzania wypożyczeniami w bibliotece. Projekt obejmuje obsługę bazy czytelników, katalogu zbiorów bibliotecznych (z uwzględnieniem autorów i kategorii literackich), a także pełen proces ewidencji wypożyczeń, zwrotów oraz historii operacji każdego użytkownika.

3.1.2 Opis procesów i więzy integralności:

Głównym procesem w systemie jest obieg książki między biblioteką a czytelnikiem.

- **Rejestracja:** Czytelnik zapisuje się do biblioteki, podając swoje dane osobowe oraz adresowe. System automatycznie rejestruje datę zapisu.
- **Katalogowanie:** Książki są kategoryzowane (np. Fantastyka, Kryminał, Literatura faktu) i przypisane do konkretnych autorów. Każdy fizyczny egzemplarz książki posiada swój unikalny identyfikator, co umożliwia śledzenie niezależnie każdego egzemplarza.
- **Wypożyczenia i Zwroty:** Proces wypożyczenia łączy czytelnika z konkretnym egzemplarzem książki w czasie. Rejestrowana jest data wypożyczenia. System zakłada konieczność ewidencji dokładnej daty zwrotu rzeczywistego, co pozwala na analizy przeterminowanych wypożyczeń i wzorców korzystania z kolekcji.
- **Statystyki (Więzy integralności):** Pola takie jak “aktualne wypożyczenia” i “suma wypożyczeń” z pierwotnego prototypu są wartościami wyliczalnymi (pochodnymi) na podstawie historii transakcji i są usuwane ze schematu normalnego.

3.1.3 Wykaz gromadzonych danych:

- **Dane czytelnika:** Imię, Nazwisko, Ulica, Kod pocztowy, Miasto, Data zapisu.
- **Dane książki:** Tytuł, Rok wydania, Dostępność.
- **Dane autora:** Imię, Nazwisko.
- **Dane kategorii:** Nazwa gatunku literackiego.
- **Dane transakcyjne:** Data wypożyczenia, Data zwrotu (rzeczywista).

3.2 Prototyp CSV

Aby zweryfikować kompletność przetwarzanych informacji, przygotowano “płaską” (nieznormalizowaną) reprezentację danych dla operacji wypożyczenia, bazującą na pierwotnych wytycznych.

```
Imie,Nazwisko,Ulica,Kod_Pocztowy,Miasto,Data_Zapisu,Tytul_Ksiazki,Imie_Autora,  
Nazwisko_Autora,Kategoria,Data_Wypozyczenia,Data_Zwrotu  
Jan,Kowalski,Kwiatowa 1,00-001,Warszawa,2024-01-15,Wiedźmin,Andrzej,Sapkowski,
```

(continues on next page)

(continued from previous page)

↪Fantastyka, 2024-03-01, 2024-03-15
 Jan, Kowalski, Kwiatowa 1, 00-001, Warszawa, 2024-01-15, Pan Tadeusz, Adam, Mickiewicz, Poezja,
 ↪2024-03-10,
 Anna, Nowak, Polna 2, 31-444, Kraków, 2023-11-05, Lśnienie, Stephen, King, Horror, 2024-02-20,
 ↪2024-03-05

3.3 Model Konceptualny (Pojęciowy)

Na podstawie analizy procesów i zebranych danych opracowano model pojęciowy, identyfikując obiekty, ich cechy oraz powiązania.

3.3.1 Zidentyfikowane encje

- **Czytelnik** - osoba zapisana do biblioteki.
- **Autor** - twórca dzieła literackiego.
- **Książka** - oferowana pozycja z inwentarza biblioteki.
- **Kategoria** - sekcja grupująca gatunkowo księgozbiór.
- **Wypożyczenie** - zdarzenie potwierdzające fakt wydania książki czytelnikowi.

3.3.2 Zdefiniowane atrybuty/własności

- **Czytelnik:** Imię, Nazwisko, Ulica, Kod pocztowy, Miasto, Data zapisu.
- **Autor:** Imię, Nazwisko.
- **Książka:** Tytuł, Rok wydania.
- **Kategoria:** Nazwa kategorii.
- **Wypożyczenie:** Data wypożyczenia, Data zwrotu.

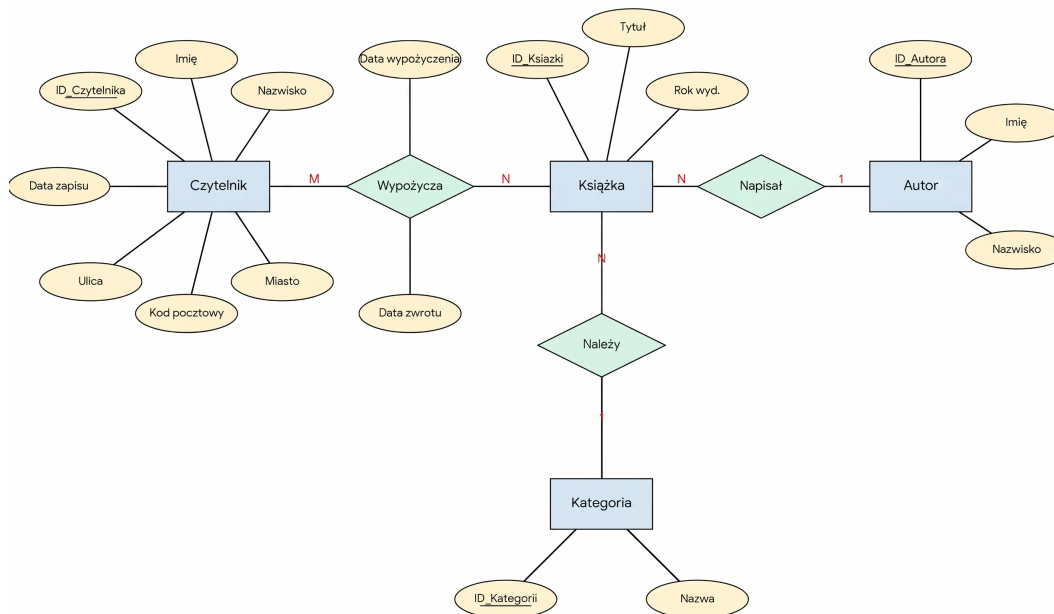
3.3.3 Opis związków

- **Autor – Książka (1:N):** Jeden autor może napisać wiele książek. (Dla uproszczenia modelu zakładamy głównego autora).
- **Kategoria – Książka (1:N):** Kategoria zawiera wiele tytułów.
- **Czytelnik – Wypożyczenie (1:N):** Czytelnik może dokonać wielu wypożyczeń w historii.
- **Książka – Wypożyczenie (1:N):** Jeden egzemplarz książki może być w swojej historii wypożyczany wielokrotnie różnym czytelnikom.

3.3.4 Identyfikacja encji słabych

Występuje relacja wiele-do-wielu (M:N) między Czytelnikiem a Książką (czytelnik czyta wiele książek, książka jest czytana przez wielu czytelników). Związek ten rozwiązywany jest przez encję asocjacyjną **Wypożyczenia**, która przechowuje wszystkie historyczne i bieżące transakcje między dwoma stronami. * **Uzasadnienie:** Byt ten nie istnieje samodzielnie w oderwaniu od konkretnego Czytelnika i konkretnej Książki.

3.3.5 Schemat w notacji Chena



3.4 Model logiczny i proces normalizacji

3.4.1 Przebieg procesu normalizacji

Krok 1: Pierwsza Postać Normalna (1NF) Wiersze są unikalne, tworzymy sztuczne klucze główne (ID_Wypozyczenia). Wartości w komórkach są atomowe. Pojawia się duża redundancja danych adresowych i książkowych.

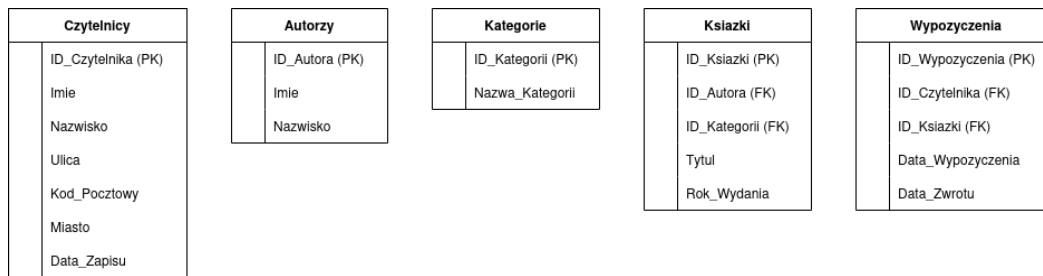
Krok 2: Druga Postać Normalna (2NF) Rozbijamy płaską strukturę względem częściowych zależności. Oddzielamy encje twarde od operacji. Tworzymy bazowe tabele: Czytelnicy oraz Książki. Powstaje tabela Wypozyczenia przechowująca klucze obce ID_Czytelnika oraz ID_Książki, a także daty transakcji. *Uwaga:* Dynamiczne atrybuty takie jak “suma wypożyczeń” z pierwotnego zadania są usuwane ze schematu, ponieważ łamią zasady normalizacji – można je obliczyć zapytaniem SQL typu COUNT(*) na zdarzeniach w tabeli Wypozyczenia.

Krok 3: Trzecia Postać Normalna (3NF) Eliminacja zależności przechodnich. Z tabeli Książki wydzielamy powtarzające się nazwy gatunków do tabeli Kategorie (klucz: ID_Kategorii) oraz dane twórców do tabeli Autorzy (klucz: ID_Autora). Każda tabela reprezentuje teraz niezależną encję o jasno określonym celu biznesowym.

3.4.2 Ostateczna struktura tabel (3NF)

- **Czytelnicy:** ID_Czytelnika (PK), Imie, Nazwisko, Ulica, Kod_Pocztowy, Miasto, Data_Zapisu
- **Autorzy:** ID_Autora (PK), Imie, Nazwisko
- **Kategorie:** ID_Kategorii (PK), Nazwa_Kategorii
- **Książki:** ID_Książki (PK), ID_Autora (FK), ID_Kategorii (FK), Tytuł, Rok_Wydania
- **Wypozyczenia:** ID_Wypozyczenia (PK), ID_Czytelnika (FK), ID_Książki (FK), Data_Wypozyczenia, Data_Zwrotu

3.4.3 Diagram ERD (Model Logiczny)



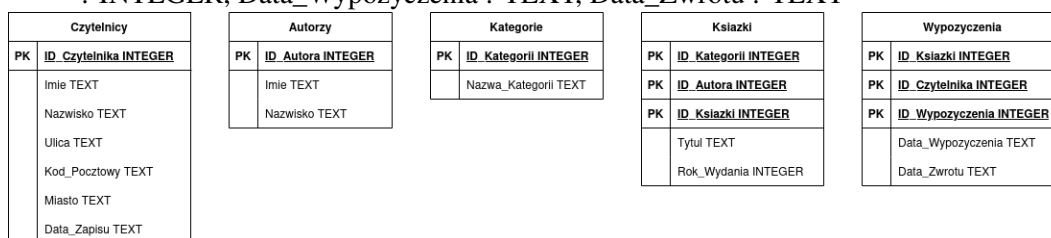
3.5 Model fizyczny bazy danych

Różnice implementacyjne między modelami fizycznymi wynikają wprost z silników RDBMS – ograniczeń typowania SQLite oraz bogatych możliwości deklaracyjnych PostgreSQL.

3.5.1 Model fizyczny dla środowiska SQLite

Z uwagi na okrojony zestaw typów, daty są mapowane jako TEXT.

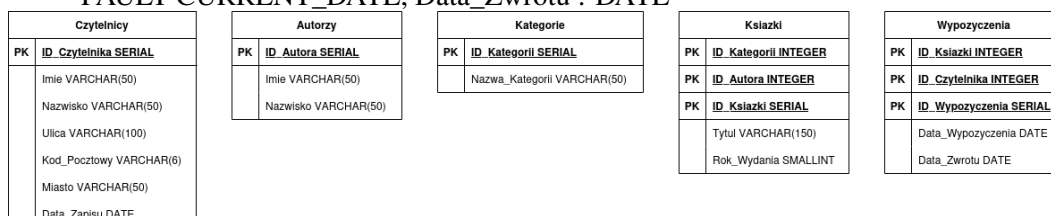
- **Czytelnicy:** ID_Czytelnika : INTEGER PRIMARY KEY, Imie : TEXT, Nazwisko : TEXT, Ulica : TEXT, Kod_Pocztowy : TEXT, Miasto : TEXT, Data_Zapisu : TEXT
- **Autorzy:** ID_Autora : INTEGER PRIMARY KEY, Imie : TEXT, Nazwisko : TEXT
- **Kategorie:** ID_Kategorii : INTEGER PRIMARY KEY, Nazwa_Kategorii : TEXT
- **Książki:** ID_Ksiazki : INTEGER PRIMARY KEY, ID_Autora : INTEGER, ID_Kategorii : INTEGER, Tytul : TEXT, Rok_Wydania : INTEGER
- **Wypożyczenia:** ID_Wypozyczenia : INTEGER PRIMARY KEY, ID_Czytelnika : INTEGER, ID_Ksiazki : INTEGER, Data_Wypozyczenia : TEXT, Data_Zwrotu : TEXT



3.5.2 Model fizyczny dla środowiska PostgreSQL

PostgreSQL umożliwia zastosowanie precyzyjnych i natywnych typów, w tym rygorystycznych typów daty (DATE) oraz optymalizacji pamięciowej dla ciągów znaków (VARCHAR).

- **Czytelnicy:** ID_Czytelnika : SERIAL PRIMARY KEY, Imie : VARCHAR(50), Nazwisko : VARCHAR(50), Ulica : VARCHAR(100), Kod_Pocztowy : VARCHAR(6), Miasto : VARCHAR(50), Data_Zapisu : DATE DEFAULT CURRENT_DATE
- **Autorzy:** ID_Autora : SERIAL PRIMARY KEY, Imie : VARCHAR(50), Nazwisko : VARCHAR(50)
- **Kategorie:** ID_Kategorii : SERIAL PRIMARY KEY, Nazwa_Kategorii : VARCHAR(50)
- **Książki:** ID_Ksiazki : SERIAL PRIMARY KEY, ID_Autora : INTEGER REFERENCES Autorzy, ID_Kategorii : INTEGER REFERENCES Kategorie, Tytul : VARCHAR(150), Rok_Wydania : SMALLINT
- **Wypożyczenia:** ID_Wypozyczenia : SERIAL PRIMARY KEY, ID_Czytelnika : INTEGER REFERENCES Czytelnicy, ID_Ksiazki : INTEGER REFERENCES Ksiazki, Data_Wypozyczenia : DATE DEFAULT CURRENT_DATE, Data_Zwrotu : DATE



ROZDZIAŁ 4: IMPLEMENTACJA FIZYCZNA I ZASILANIE BAZY DANYCH

Autorzy

1. Paweł Łoćwin
2. Paweł Łosowski

4.1 Definicja fizyczna bazy danych (Zadanie 1)

Na podstawie opracowanych wcześniej modeli logicznych przygotowano skrypty DDL (Data Definition Language) dla dwóch docelowych silników relacyjnych baz danych: PostgreSQL oraz SQLite. Kody te przygotowały solidną fundamentę dla skutecznego przechowywania i zarządzania danymi biblioteki.

4.1.1 Skrypt dla środowiska PostgreSQL

Środowisko PostgreSQL pozwala na wykorzystanie zaawansowanych, natywnych typów danych oraz rygorystyczne egzekwowanie więzów integralności. W poniższym skrypcie kluczowym elementem jest użycie więzów klucza obcego (FOREIGN KEY) w celu zapewnienia spójności referencyjalnej między tabelami.

```
-- Fragment kodu definicji kluczowych tabel (PostgreSQL)
CREATE TABLE Ksiazki (
    ID_Ksiazki SERIAL PRIMARY KEY,
    ID_Autora INTEGER REFERENCES Autorzy(ID_Autora),
    ID_Kategorii INTEGER REFERENCES Kategorie(ID_Kategorii),
    Tytul VARCHAR(150) NOT NULL,
    Rok_Wydania SMALLINT
);

CREATE TABLE Wypozyczenia (
    ID_Wypozyczenia SERIAL PRIMARY KEY,
    ID_Czytelnika INTEGER REFERENCES Czytelnicy(ID_Czytelnika),
    ID_Ksiazki INTEGER REFERENCES Ksiazki(ID_Ksiazki),
    Data_Wypozyczenia DATE DEFAULT CURRENT_DATE,
    Data_Zwrotu DATE
);
```

4.1.2 Skrypt dla środowiska SQLite

W silniku SQLite struktura ulega pewnemu uproszczeniu z powodu mniejszej rygorystyczności typowania oraz bardzo ograniczonej liczby wbudowanych klas typów danych (np. brak odrębnego typu daty). Pomimo ograniczeń, SQLite pozostaje niezawodnym rozwiązaniem do prototypowania i mniejszych aplikacji.

```
-- Fragment kodu definicji kluczowych tabel (SQLite)
CREATE TABLE Wypozyczenia (
    ID_Wypozyczenia INTEGER PRIMARY KEY AUTOINCREMENT,
    ID_Czytelnika INTEGER,
    ID_Ksiazki INTEGER,
    Data_Wypozyczenia TEXT,
```

(continues on next page)

(continued from previous page)

```
Data_Zwrotu TEXT,
FOREIGN KEY(ID_Czytelnika) REFERENCES Czytelnicy(ID_Czytelnika),
FOREIGN KEY(ID_Ksiazki) REFERENCES Ksiazki(ID_Ksiazki)
);
```

4.2 Mechanizm zasilania bazy danych (Zadanie 2)

Posiadając gotową strukturę bazy, należało wyposażać ją w dane demonstracyjne. Przygotowano zbiór danych testowych, które zostały poddane wsadowemu ładowaniu (batch import).

4.2.1 Formatyzacja danych (Pliki CSV)

Dane do importu przygotowano w niezależnym formacie płaskim CSV (Comma-Separated Values). Takie podejście pozwala na łatwą edycję danych poza systemem bazodanowym. Poniżej zaprezentowano wybrane kategorie książek przygotowane w formacie CSV.

```
Nazwa_Kategorii
Fantastyka
Kryminał
Literatura faktu
Poezja
Horror
```

4.2.2 Implementacja mechanizmu importu

Do zaimportowania powyższych danych wybrano mechanizm wprowadzania wielowartościowego oparty na technice **INSERT (multi-value/batch)**. Rozwiązanie zrealizowano z wykorzystaniem języka Python wraz z biblioteką `psycopg` do komunikacji z bazą PostgreSQL.

Wybór tego mechanizmu podyktowany jest kompromisem między uniwersalnością a wydajnością. Użycie metody `executemany()` na obiekcie kursora bazy danych pozwala na szybkie wstawienie wielu rekordów bez konieczności wykonywania zapytania dla każdego wiersza osobno.

Poniżej przedstawiono fragment skryptu aplikacyjnego odpowiedzialnego za odczyt z pliku i zasilenie tabel:

```
import csv
import psycopg

# 1. Wczytywanie danych z pliku CSV do struktury iterowalnej (listy krotek)
def wczytaj_dane_csv(sciezka_pliku):
    dane = []
    with open(sciezka_pliku, mode='r', encoding='utf-8') as plik:
        csv_reader = csv.reader(plik)
        next(csv_reader) # Pominięcie wiersza nagłówka
        for wiersz in csv_reader:
            dane.append(tuple(wiersz))
    return dane

# 2. Właściwy proces wsadowego importu do bazy PostgreSQL
def importuj_do_postgres(kategorie_do_importu, db_config):
    try:
        # Użycie Context Managera dla bezpiecznego zarządzania transakcjami
        with psycopg.connect(**db_config) as conn:
            with conn.cursor() as cursor:
```

(continues on next page)

(continued from previous page)

```
# Definicja sparametryzowanego zapytania zapobiegającego SQL Injection
zapytanie_sql = "INSERT INTO Kategorie (Nazwa_Kategorii) VALUES (%s)"

# Wsadowe wywołanie zapytań (batch insert)
cursor.executemany(zapytanie_sql, kategorie_do_importu)

print(f"Pomyślnie zaimportowano {len(kategorie_do_importu)} rekordów.
↪")

except Exception as e:
    print(f"Wystąpił błąd operacji wsadowej: {e}")
```

4.3 Podsumowanie

Niniejszy rozdział wieńczy etap projektowania i wdrażania struktury relacyjnej bazy danych dla systemu zarządzania biblioteką. Poprzez transformację modelu logicznego do postaci fizycznej osiągnięto kompletne i funkcjonalne rozwiązanie.

Pierwsza część rozdziału koncentrowała się na **Definicji fizycznej bazy danych (Zadanie 1)**, gdzie dokonano implementacji schematów dla dwóch wiodących silników relacyjnych: PostgreSQL i SQLite, stanowiących praktyczną alternatywę w zależności od wymagań skalowalności oraz zasobów dostępnych w środowisku produkcyjnym. Druga część rozdziału poświęcona została **Mechanizmowi zasilania bazy danych (Zadanie 2)**. Dane demonstracyjne przygotowano w formacie CSV, który zapewnia elastyczność i łatwość edycji zarówno dla administratorów baz danych, jak i dla użytkowników końcowych systemu.

Automatyzacja procesu inicjalizacji systemu za pomocą skryptów Python stanowi znaczący wkład w praktyczność rozwiązania, umożliwiając łatwe replikowanie i testowanie bazy danych w różnych środowiskach bez konieczności wykonywania repetycyjnych czynności manualnych.

Niniejszy projekt stanowi kompletne studium cyklu życia relacyjnej bazy danych – od teoretycznego modelowania (Rozdziały 1-3) poprzez praktyczną implementację (Rozdział 4) – demonstrując całkowity proces od koncepcji do produkcyjnego systemu zarządzania danymi.